基于验证技能接口与执行反馈的 LLM 辅助移动机器人任务规划

候选人面试作业

摘要

本文介绍一个将自然语言指令转换为少量、经过验证的高层机器人技能调用的单机器人系统。系统运行于 ROS 2 Humble 与 Webots R2025a，部署在 Windows 主机上的 WSL2 中。任务级规划器只能从固定的三项技能白名单中选择；多层验证器在每个调用执行前进行检查；有界恢复模块观察每个结构化执行结果，并在重试、避障恢复、带执行反馈的一次性 LLM 重规划、安全停止上报之间做出决策。实验表明：系统能以 0.049 m 的位置误差到达目标，能可靠执行多航点任务，能安全拒绝越界与未知技能计划，并能给出“LLM 收到超时反馈后重新生成计划”的闭环执行轨迹。

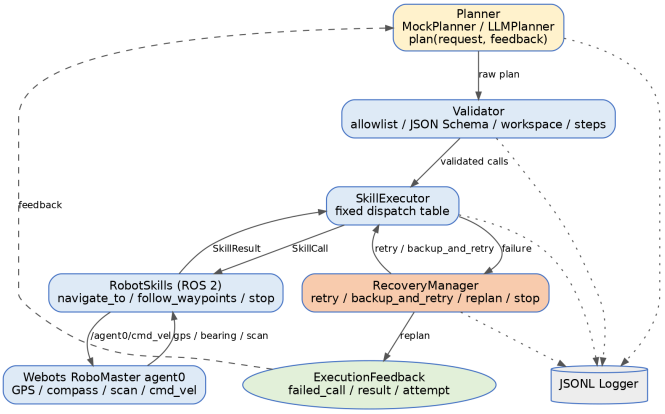


图 1: 闭环系统架构。虚线为 LLM 规划器用于有界重规划的执行反馈通道。

1 引言

大语言模型可以将自然语言翻译为结构化工具调用，但直接在机器人上执行其输出并不安全。本作业要求：LLM 只能**选择**已有的高层技能，不得直接发布速度指令，也不得执行生成的代码。本文的设计目标是可靠性、显式技能契约、严格验证与可复现实验。

系统遵循如下闭环：`Plan` → `Validate` → `Execute` → `Observe` → `Recover`。主要贡献包括：

- 具有固定白名单与 JSON Schema 契约的验证式技能接口；
- `navigate_to`、`follow_waypoints`、`stop` 三个技能的 ROS 2 实现；
- 集中管理规划、验证、执行、观察与恢复的闭环任务代理 `TaskAgent`；
- 规则规划器与可消费结构化执行反馈进行有界重规划的 OpenAI 兼容 LLM 规划器；
- 针对越界地点、未知技能、超时与障碍阻塞的有界恢复；
- 可复现的 WSL2/Webots 环境与 JSONL 执行日志。

2 系统架构

图 1 给出系统架构。规划器生成原始计划（`SkillCall[]`）；验证器在机器人执行任何动作之前检查计划；只有通过验证的调用才会到达技能执行器；执行器调用 `RobotSkills` ROS 2 节点；该节点发布速度指令，并从 Webots 读取位置、航向与激光反馈。每个技能返回结构化 `SkillResult`；执行失败时，恢复模块决定重试、后退重试、带 `ExecutionFeedback` 的重规划、或停止上报。所有事件追加写入 JSONL 日志。

2.1 模块划分

仓库将任务级代理与 ROS 2 技能层分离：

- `agent/task_agent.py`：负责 Plan–Validate–Execute–Observe–Recover 闭环；
- `agent/planner.py`：规则规划器与可选 LLM 规划器；
- `agent/registry.py`：机器可读技能注册表；
- `agent/validator.py`：六层验证；
- `agent/executor.py`：到三项技能的固定分发表；
- `agent/recovery.py`：有界恢复决策；
- `agent/models.py`：`SkillCall`、`SkillResult`、`ValidationResult`、`ExecutionFeedback`；

表 1： 技能接口摘要。

技能	契约
`navigate_to`	参数：x, y, θ；成功条件：位置误差 $\le 0.05\,\mathrm{m}$ 且航向误差 $\le 0.10\,\mathrm{rad}$；超时：30 s
`follow_waypoints`	参数：航点列表；失败时返回最后到达的航点下标；超时：每个航点 30 s
`stop`	发布零速度；成功条件：已发布零速度；超时：2 s

- `agent/logger.py`：JSONL 事件日志；
- `robot_skills/skills.py`：技能的 ROS 2 实现。

2.2 技能接口

表 1 概括三项技能。注册表是唯一事实来源：同一份定义既生成验证器使用的 JSON Schema，也生成暴露给 LLM 的工具列表。

3 实现细节

3.1 ROS 2 技能层

`RobotSkills` 节点订阅 `/agent0/gps`（`PointStamped`）获取位置，订阅 `/agent0/compass/bearing`（`FloatStamped`）获取航向，并在 `/agent0/cmd_vel` 上发布 `Twist` 速度指令。Webots 罗盘方位角以顺时针为正，而 ROS 偏航角以逆时针为正，因此节点按如下公式转换：

$$\psi = \mathrm{normalize}(-\mathrm{radians}(\mathrm{bearing})). \tag{1}$$

3.2 全向 `navigate_to`（PID）

RoboMaster 底盘为全向底盘。控制器无需先旋转再直行，而是直接在车体系中平移，仅在最后对航向进行原地旋转校正：

$$e_x = x_d - x, \qquad e_y = y_d - y, \tag{2}$$

$$v_x^w = \mathrm{clamp}\big(k_p e_x + k_i \textstyle\int e_x dt + k_d \frac{de_x}{dt}, -v_{\max}, v_{\max}\big), \tag{3}$$

$$v_y^w = \mathrm{clamp}\big(k_p e_y + k_i \textstyle\int e_y dt + k_d \frac{de_y}{dt}, -v_{\max}, v_{\max}\big), \tag{4}$$

$$v_x^b = \cos\psi\, v_x^w + \sin\psi\, v_y^w, \tag{5}$$

$$v_y^b = -\sin\psi\, v_x^w + \cos\psi\, v_y^w. \tag{6}$$

当位置误差大于容差时，节点发布 $(v_x^b, v_y^b, 0)$；否则发布 $(0, 0, \omega)$，其中 $\omega = \mathrm{clamp}\big(k_p^\theta e_\theta + k_i^\theta \int e_\theta dt + k_d^\theta \frac{de_\theta}{dt}, -\omega_{\max}, \omega_{\max}\big)$。成功条件为

$$\sqrt{e_x^2 + e_y^2} \le 0.05\,\mathrm{m} \quad \text{且} \quad |e_\theta| \le 0.10\,\mathrm{rad}. \tag{7}$$

超时后节点停止机器人并返回 `success=false, reason="timeout"`。

3.3 `follow_waypoints` 与 `stop`

`follow_waypoints` 依次调用 `navigate_to` 并维护 `last_reached_index`。若某个航点失败，节点停车并返回该下标。`stop` 发布零速度并返回结构化结果。

3.4 障碍安全层

`navigate_to` 同时订阅 `/scan`。在发布平移指令前，它检查意图运动方向 30 度锥角内的最小有效距离。若该距离不超过 0.35 m，技能停车并返回 `success=false, reason="obstacle_blocked"`。任务级恢复随后执行后退、旋转并重试一次。

3.5 闭环任务代理

`TaskAgent` 拥有完整执行循环。它首先请求规划器生成计划并记录原始计划；验证器在运动前检查计划，无效则拒绝。对每个通过验证的调用，执行器调用对应技能并记录观察到的 `SkillResult`。成功的调用被记录并继续执行。失败时，`RecoveryManager` 选择有界动作：

- `retry`：超时后重新执行同一调用一次；
- `backup_and_retry`：障碍阻塞后后退、旋转并重试一次；
- `replan`：将 `ExecutionFeedback` 发送给 LLM 规划器并请求新计划（仅当规划器支持反馈且重规划预算未耗尽）；
- `stop_and_report`：停车并返回失败。

重规划产生的新计划必须在任何机器人动作前重新通过验证。若规划器返回相同计划，代理安全停止而不会循环。重规划次数由 `--max-replans` 限定（默认 1）。

3.6 验证层

验证器共六层：计划长度、固定白名单、JSON Schema、工作空间边界、前置条件、重试/重规划次数限制。允许的工作空间为 $x, y \in [-1.5, 1.5]\,\mathrm{m}$。表 2 列出各层及对应失败原因。

表 2: 验证层及产生的失败原因。

验证层	失败原因
计划长度	`too_many_steps`
白名单	`unknown_skill`
JSON Schema	`schema_invalid`
工作空间边界	`target_out_of_workspace`
前置条件	`precondition_failed`
重试/重规划限制	`stop_and_report`

4 实验与结果

4.1 实验平台

实验平台为 Windows 11 + WSL2 Ubuntu 22.04 + ROS 2 Humble + Webots R2025a。GPU 为 AMD Radeon RX 9070 XT（16 GB）。WSLg 提供基于 Mesa D3D12 驱动的硬件加速 OpenGL 4.2 上下文：`glxinfo` 报告渲染器 `D3D12 (AMD Radeon RX 9070 XT)` 且 `Accelerated: yes`；`glxgears` 约 140 FPS。

4.2 任务级回归测试

表 3 给出闭环重构后的完整回归结果。成功用例以 0.0494 m 的位置误差到达 storage。多航点用例到达全部航点并报告 `last_reached_index=1`。越界地点在 mock 规划器下被工作空间层拒绝；真实 LLM 对越界请求返回空工具列表，验证器在执行前拒绝。未知技能被白名单层拒绝。强制超时用例重试一次后被安全停止并上报。障碍恢复用例在后退 + 旋转重试后成功；持续障碍用例则停车并上报目标被阻塞。真实云 LLM 规划器使用 OpenAI 兼容端点与 gpt-5.5 模型，正确生成了 `navigate_to` 以及 `follow_waypoints + stop` 计划。

关键新结果是 R12。LLM 首先规划前往 charging_station。在连续两次超时观测之后，恢复层将结构化 `ExecutionFeedback` 发回 LLM。LLM 返回新的经过验证的计划，改为前往 storage；由于超时仍被强制为 1 秒，该计划最终被安全停止并上报。因此 JSONL 日志包含完整序列：`raw_plan` → `skill_result` → `recovery_decision` → `feedback_to_planner` → `replanned_plan` → `validation` → `skill_result` → `final_outcome`。

4.3 手动控制与标定

表 4 给出参考 `wheels_controller` 的观测响应。该控制器使用了错误的轮半径常量（0.26，而 URDF 中为 0.06），因此技能层采用闭环反馈而非开环定时。

5 讨论

5.1 安全性

LLM 从不获得直接执行器权限。它只能从三项白名单技能中选择，且每个提议调用在执行前都经过验证器。执行器将技能名映射到具有预定义签名的 Python 方法；任何生成的 Python、Shell 或 ROS 命令都不会被执行。越界目标、未知技能、参数错误、步数超限与空计划均被拒绝。超时或障碍阻塞最终都会以零速度指令和结构化失败报告结束。

5.2 执行反馈

重构后的代理使执行反馈显式且有界。LLM 只看到任务级信息：原始请求、失败调用、结构化结果、恢复消息与已完成工作；它从不接触原始传感器数据。反馈路径由规则触发，因此系统在使用 mock 规划器时保持确定性和可测试性，同时支持真实 LLM 对比实验。

5.3 可复现性

系统可依据仓库与 `docs/environment.md` 中的环境说明复现。所需依赖包括 ROS 2 Humble、Webots R2025a、`webots_ros2` 2025.0.0 与参考包 `webots_ros2_robomaster` 的世界资源。Webots 资源缓存与参考 `protos/meshes` 目录已在文档中说明。生成的 JSONL 日志作为示例提交。

5.4 局限性

最终配置在 WSL2 中通过 WSLg 运行 Webots。WSLg 存在一定的 D3D12 转换与 Xwayland 合成开销，流畅度不如可选的 Windows 原生 Webots 模式，但足以完成本作业。参考环境没有 `/agent0/odom` 话题，因此技能层使用 `/agent0/gps` 获取位置、`/agent0/compass/bearing` 获取航向。参考 `wheels_controller` 的轮半径常量有误，因此必须采用闭环控制。当前 LLM 重规划为一次性、规则触发，尚未对原始计划与重规划计划进行除“完全相同检测”之外的更深比较。

表 3: 闭环代理的任务级回归结果。

ID	指令	规划器/模式	结果	技能调用	恢复动作
R1	Go to storage.	mock / direct	成功，18.58 s，0.0494 m	1	无
R2	Visit storage, then workbench, and stop.	mock / direct	成功，38.27 s	2	无
R3	Go to storage.	mock / map (A*)	成功，17.73 s，`map_reached`	1	无
R4	Go to space_station.	mock	拒绝	0	中止，`target_out_of_`
R5	Go to entrance and fly.	mock	拒绝	0	中止，`unknown_skill`
R6	Go to charging_station，强制 1 s 超时	mock / direct	失败	2	重试一次后停止上报
R7	Go to storage，障碍后消失	mock / direct	成功，0.0500 m	2	后退、旋转、重试一次
R8	Go to storage，障碍持续存在	mock / direct	失败	2	后退、旋转、停止上报
R9	Go to storage.	真实 LLM / direct	成功，16.43 s，0.0499 m	1	无
R10	Visit storage, then workbench, and stop.	真实 LLM / direct	成功，38.70 s	2	无
R11	Go to space_station.	真实 LLM	拒绝	0	LLM 返回空计划，中止
R12	Go to charging_station，强制 1 s 超时	真实 LLM / direct	失败	4	重试，**带反馈重规划**，

表 4: 观测到的指令响应。

指令	观测运动	近似响应
`linear.x = 1.0`	2 s 内移动 0.20 m	0.10 m/s
`linear.y = 1.0`	2 s 内移动 0.22 m	0.11 m/s
`angular.z = 5.0`	2 s 内转动 127°	1.11 rad/s

5.5 后续工作

未来工作包括：让 LLM 规划器基于完整执行轨迹与显式失败语义进行推理；使用 `/scan` 进行动态避障；增加专用里程计话题；以及 Docker/WSL 一键化环境。

6 结论

本文介绍了一个可靠、经过验证的 LLM 辅助任务规划系统。系统将规划、验证、执行、观察与恢复分离；强制固定白名单；每个技能返回结构化结果；并通过有界执行反馈将执行结果送回 LLM 规划器。实验证明了位姿到达、多航点执行、基于地图的 A* 导航、越界与未知技能拒绝、障碍恢复，以及真实 LLM 在观测到超时后进行重规划的完整闭环轨迹。

参考文献

[1] ROS 2 Humble 文档。https://docs.ros.org/en/humble/

[2] Cyberbotics Webots。https://cyberbotics.com/

[3] `webots_ros2`。https://github.com/cyberbotics/webots_ros2

[4] `webots_ros2_robomaster`。https://github.com/MohismLab/webots_ros2_robomaster

[5] Microsoft WSLg。https://github.com/microsoft/wslg

[6] Mesa D3D12 驱动。https://docs.mesa3d.org/drivers/d3d12.html

[7] OpenAI 兼容函数调用。https://platform.openai.com/docs/guides/function-calling